

Reviewer Pack — Tamari Rotation Rank of Canonical DT Lower-Bounds XOR-Lifted...

Entry b045e08ff6c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 19:17:16 UTC

Tamari Rotation Rank of Canonical DT Lower-Bounds XOR-Lifted F2-Rank

Entry ID: b045e08ff6c7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 19:17:16 UTC

1. Conjecture

Field A (mathematical branch): Tamari lattice combinatorics — the order-theoretic Pallo rank function $\text{rrd}(T) = \sum \text{over internal nodes } v \text{ of } |L(v)|$ (size of v 's left-subtree leaf set), in the Sleator-Tarjan-Thurston / Pallo-Bonnin / Pournin / Chapoton tradition on Catalan-counted binary trees. An arXiv search for 'Tamari lattice' AND ('communication complexity' OR 'lifting theorem' OR 'decision tree complexity') returns no direct hits and <5 adjacent papers — this corner of Catalan/order combinatorics has essentially no presence in CC / lifting literature. **Field B** (complexity object): F2-rank of the XOR-lifted communication matrix $M_{\{f \text{ xor}\}}$ for Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$, defined by $M[a,b] = f(a \text{ XOR } b)$, in the Raz-McKenzie / Goos-Pitassi-Watson lifting framework. $\log_2 \text{rank}_{\{F2\}}(M)$ is the canonical algebraic lower bound for deterministic two-party $\text{CC}^D(M_{\{f \text{ xor}\}})$, the XOR-function CC studied since Lee-Shraibman, Hatami-Hosseini-Lovett, and the Log-Rank Conjecture.

Statement:

For a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, let T_{f^*} be the canonical minimum-leaf decision tree obtained by a deterministic DP on partial assignments: at each restriction state, branch on the smallest-index variable whose two children have strictly smaller leaf totals, with lex tie-breaking on serialized (left-shape, right-shape) tuples. Let $\text{rrd}(T_f) = \sum \text{over internal nodes } v \text{ of } |L(v)|$, the Pallo right-rotation rank in the Tamari lattice. Define $M = M_{\{f \text{ xor}\}}$, the 2^n by 2^n matrix $M[a,b] = f(a \text{ XOR } b)$. Conjecture: there is a universal constant $c \geq 1/4$ such that for every f with $\text{rrd}(T_f) \geq 4$, $\log_2(\text{rank}_{\{F2\}}(M)) \geq c * \log_2(1 + \text{rrd}(T_f))$. A single f with $\text{rrd}(T_f) \geq 4$ but $\log_2(\text{rank}_{\{F2\}}(M)) < 0.25 * \log_2(1 + \text{rrd}(T_{f^*}))$ refutes the conjecture.

Rationale (proposer's reasoning):

Raz-McKenzie / GPW lifting passes the STRUCTURE of a decision tree T_f into the rectangle/block structure of $M_{\{f \text{ o } g\}}$; the Pallo rank $\text{rrd}(T_f)$ is the most natural Catalan-order measure of how 'right-comb-asymmetric' that block structure is, in a regime never previously connected to CC. The invariant is intrinsic to the chosen minimum tree T_f (a non-polynomial selection over restrictions), so it is not a polynomial in truth-table entries and the construction does not extend to polynomial extensions of the Boolean ring — sidestepping algebrization. The bridge respects LIFTING's safe directions (no relativization, no Karp-Lipton) and targets a CC-rank object, not a P/poly truth-table predicate, so natural-proofs does not bite.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 89480b76d8a0a4b4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For n in $\{4,5,6,7\}$ with 30 seeds each (120 total), compute ratio $r = \log_2(\text{rank_F2}(M)) / \log_2(1 + \text{rrd}(T_f^*))$ on instances with $\text{rrd} \geq 4$. Support requires median $r \geq 0.25$ at every n AND every qualifying seed has $r \geq 0.25$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tamari lattice rotation distance decision tree complexity Boolean function - Pallo rank Catalan binary tree XOR function communication complexity F2 rank - decision tree leaf complexity lifting theorem log-rank XOR-function GF(2) rank

Top relevant hits considered: - [s2:10.1007/s00037-025-00276-5] Lifting Dichotomies - [s2:7250a91643b33e65c107da27b0c02900b7611476] Lifting Theorems for Equality

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import json
from collections import defaultdict

def build_decision_tree(f, n):
    # Build the canonical decision tree T_f* using DP on partial assignments
    # Each state is represented as a tuple of (var, val) pairs
    memo = {}

    def dp(state):
        if state in memo:
            return memo[state]
```

```

        return memo[state]

    # Check if all variables are fixed
    fixed_vars = {var for var, _ in state}
    if len(fixed_vars) == n:
        # Leaf node: evaluate the function on the fixed assignment
        assignment = [0] * n
        for var, val in state:
            assignment[var] = val
        return (1, (assignment, assignment)) # (min_leaf_count, (left_serial, right_serial))

    # Find the smallest unfixed variable
    for var in range(n):
        if var not in fixed_vars:
            break

    # Recursively build the left and right subtrees
    left_state = state + ((var, 0),)
    right_state = state + ((var, 1),)
    left_count, left_serial = dp(left_state)
    right_count, right_serial = dp(right_state)

    # Choose the child with the smallest min_leaf_count, with lex tie-break
    if left_count < right_count:
        min_count = left_count
        serial = (left_serial, right_serial)
    elif right_count < left_count:
        min_count = right_count
        serial = (right_serial, left_serial)
    else:
        # Tie-break on serialized tree shapes
        if left_serial < right_serial:
            min_count = left_count
            serial = (left_serial, right_serial)
        else:
            min_count = right_count
            serial = (right_serial, left_serial)

    memo[state] = (min_count + 1, serial)
    return memo[state]

_, tree = dp(())
return tree

def compute_rrd(tree):
    # Compute the right-rotation rank (rrd) of the decision tree
    if isinstance(tree, tuple) and len(tree) == 2 and isinstance(tree[0], tuple) and isinstance(tree[1], tuple):
        left_leaves = compute_rrd(tree[0])
        right_leaves = compute_rrd(tree[1])
        return left_leaves + right_leaves + abs(left_leaves - right_leaves)
    else:
        return 1

def construct_matrix(f, n):
    # Construct the matrix M[a,b] = f(a XOR b)
    size = 1 << n

```

```

matrix = []
for a in range(size):
    row = []
    for b in range(size):
        xor = a ^ b
        row.append(f[xor])
    matrix.append(row)
return matrix

def rank_f2(matrix):
    # Compute the rank of the matrix over F2 using Gaussian elimination
    if not matrix:
        return 0

    rows = len(matrix)
    cols = len(matrix[0])
    rank = 0

    for col in range(cols):
        # Find the pivot row
        pivot = -1
        for row in range(rank, rows):
            if matrix[row][col] == 1:
                pivot = row
                break

        if pivot == -1:
            continue

        # Swap the current row with the pivot row
        matrix[rank], matrix[pivot] = matrix[pivot], matrix[rank]

        # Eliminate this column in all other rows
        for row in range(rows):
            if row != rank and matrix[row][col] == 1:
                for c in range(col, cols):
                    matrix[row][c] ^= matrix[rank][c]

        rank += 1

    return rank

def run_trial(seed):
    random.seed(seed)
    n = random.choice([4, 5, 6, 7])
    f = [random.randint(0, 1) for _ in range(1 << n)]

    # Build the decision tree and compute rrd
    tree = build_decision_tree(f, n)
    rrd = compute_rrd(tree)

    # Construct the matrix and compute its rank over F2
    matrix = construct_matrix(f, n)
    rank = rank_f2(matrix)

    # Compute the ratio

```

```

if rrd >= 4:
    ratio = math.log2(rank) / math.log2(1 + rrd)
else:
    ratio = 1.0 # Default value when rrd < 4

# Determine if the conjecture holds
conjecture_holds = ratio >= 0.25 if rrd >= 4 else True
counterexample = f"rrd={rrd}, ratio={ratio}" if not conjecture_holds and rrd >= 4 else ""

return {
    "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    metric_values = []
    conjecture_holds_counts = 0
    total_instances = 0
    first_failing_seed = None

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps({'seed': seed, **result})}")
        metric_values.append(result["metric_value"])
        if result["conjecture_holds"]:
            conjecture_holds_counts += 1
        total_instances += result["instances_tested"]
        if not result["conjecture_holds"] and first_failing_seed is None:
            first_failing_seed = seed

    mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0.0
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_counts / len(seeds) if seeds else 0.0

    if first_failing_seed is not None:
        print(f'RESULT: FALSIFIED counterexample="{result["counterexample"]}" first_failing_seed={first_failing_seed}')
    elif support_fraction >= 0.8:
        print(f'RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}')
    else:
        print('RESULT: INCONCLUSIVE reason=insufficient_support')

```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	0.8557702926201601	□	
23	0.996285877295627	□	
37	0.7339516263546781	□	
53	0.7929594526822421	□	
71	0.996285877295627	□	
89	0.9786021684729042	□	

Seed	Metric value	Holds?	Counterexample
103	0.9983986747235201	□	
127	0.7339516263546781	□	
149	0.9786021684729042	□	
167	0.7929594526822421	□	
191	0.9911993158528026	□	
211	0.8302382310796892	□	
233	0.9911993158528026	□	
257	0.8302382310796892	□	
277	0.9786021684729042	□	
311	0.8302382310796892	□	
347	0.8302382310796892	□	
389	0.996285877295627	□	
421	0.8302382310796892	□	
463	0.9911993158528026	□	
503	0.7929594526822421	□	
547	0.7339516263546781	□	
593	0.996285877295627	□	
631	0.8302382310796892	□	
677	0.7339516263546781	□	
727	0.9911993158528026	□	
773	0.9911993158528026	□	
821	0.8557702926201601	□	
877	0.7929594526822421	□	
929	0.7929594526822421	□	

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	0.8822973003045711
metric_std	0.10139977132925602
metric_ci95_half	0.03702596138859942
metric_min	0.7339516263546781
metric_max	0.9983986747235201
support_fraction	1.0

7. Test stdout (last 2KB)

```

ces_tested": 1, "conjecture_holds": true, "counterexample": ""}
TRIAL: {"seed": 503, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.79295
TRIAL: {"seed": 547, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.73395
TRIAL: {"seed": 593, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.99628
TRIAL: {"seed": 631, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.83023
TRIAL: {"seed": 677, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.73395
TRIAL: {"seed": 727, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.99119
TRIAL: {"seed": 773, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.99119
TRIAL: {"seed": 821, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.85577
TRIAL: {"seed": 877, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.79295
TRIAL: {"seed": 929, "metric_name": "log2(rank_F2(M)) / log2(1 + rrd(T_f*))", "metric_value": 0.79295
RESULT: SUPPORTED mean=0.8822973003045711 std=0.09969545211010329 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

Selection bias / missed adversarial case: random seeds almost certainly never hit the parity function $f(x)=x_1 \oplus \dots \oplus x_n$, which is the classical adversarial input for XOR-lifted matrices. For parity, $M[a,b]=\text{parity}(a) \oplus \text{parity}(b)$ has F2-rank exactly 2 (it equals $p \cdot 1^T + 1 \cdot p^T$), so $\log_2(\text{rank})=1$. Meanwhile the canonical DT must split on every variable (each split strictly reduces leaf count), giving $\text{rrd} = \sum_d 2^{d-2}(n-d) = n \cdot 2^{n-1}$; for $n=4$ this is 64, so $\log_2(65) \approx 6.02$ and the ratio is $\approx 0.166 < 0.25$, refuted

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Random-seed sampling reports mean ratio 0.88 with min 0.73, but the critic identifies a structured adversarial case (parity function) where $\text{rrd} \sim n \cdot 2^n$ while F2-rank of the XOR-lift is 2, yielding ratio $\approx 0.166 < 0.25$ for $n=4$ — not probed by the random sample. Per the strict rule, an unrefuted CHALLENGE downgrades the verdict. | next: Explicitly evaluate the conjecture on the parity function $f(x)=x_1 \oplus \dots \oplus x_n$ for n in $\{4,5,6,7\}$ and other low-F2-rank structured functions (e.g., affine, inner product)

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	398734
2	preregistration	claude_max	opus	0	0	7420
3	novelty	claude_max	opus	0	0	5049
4	novelty	claude_max	opus	0	0	6969
5	test_gen	mistral	codestral-latest	0	0	318065
6	critic	claude_max	opus	0	0	31921
7	judge	claude_max	opus	0	0	6017

Totals: 0 input tokens, 0 output tokens, $\sim \$0.0000$ cost, 774175 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 1}

(full prompt+response transcripts available in `research/audit/b045e08ff6c7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b045e08ff6c7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b045e08ff6c7.tar.gz` (if generated)